GRAVITY

FAMILY SAFETY PLATFORM

Full System Architecture Document

Technical Reference — Version 2.0

Date: 20 June 2026

Repository: github.com/kamaralam1984/gravitypro

Domain: gravitypro.kvlbusinesssolutions.com

Stack: Node.js + Express 5 · PostgreSQL + PostGIS · React + Vite · Expo 54

GRAVITY — Full System Architecture Document

Product: Gravity Family Safety Platform
Version: 2.0
Date: 20 June 2026
Repository: github.com/kamaralam1984/gravitypro
Domain: gravitypro.kvlbusinesssolutions.com

Table of Contents

---

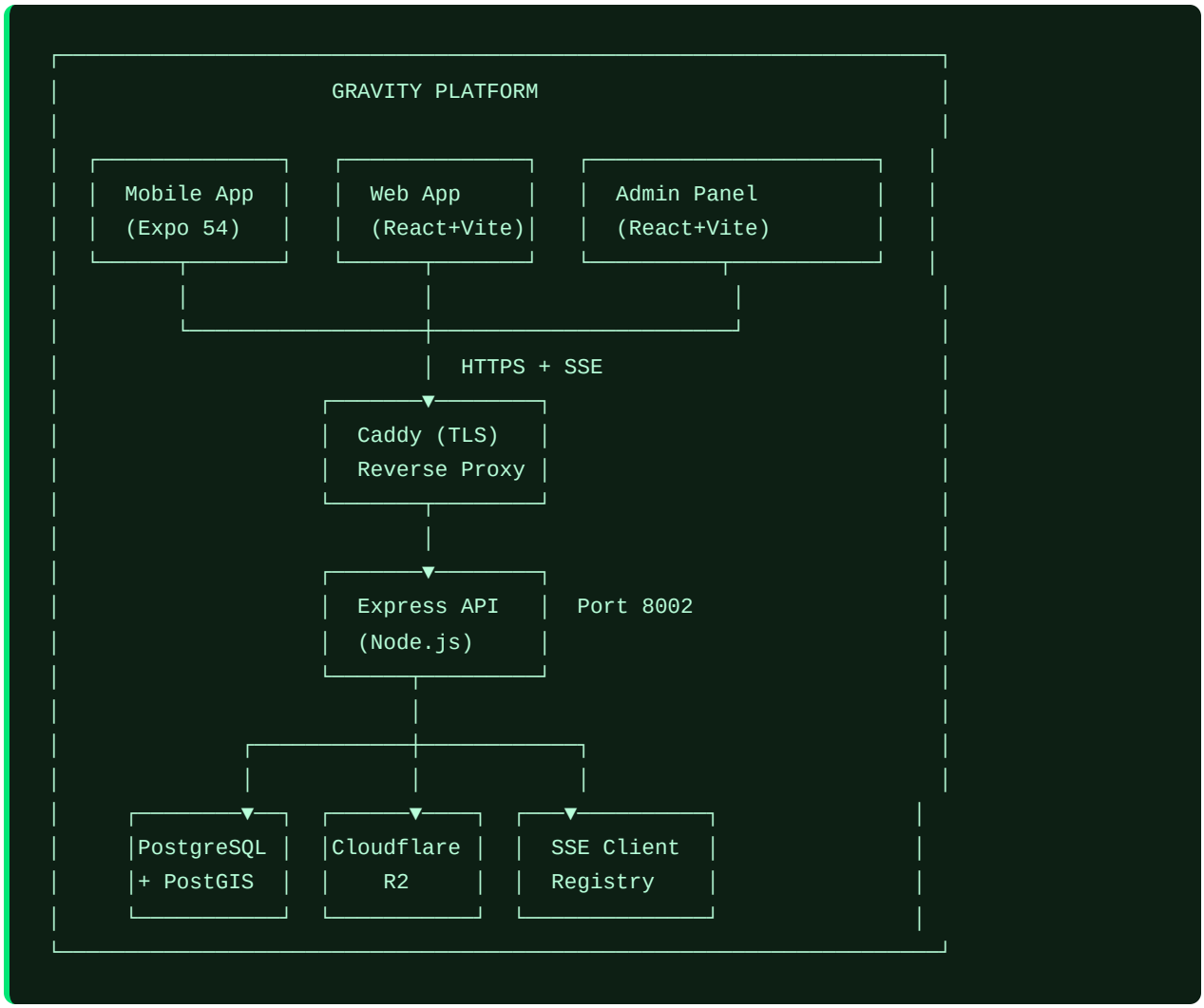
1. Executive Summary

Gravity is a **family safety and real-time location tracking platform** designed to help families stay connected and safe. The platform enables parents to monitor the live location of family members, define safe zones (geofences), receive instant SOS alerts, and manage all family groups through a premium web interface and mobile app.

Core Capabilities

Capability	Description
Live Location Tracking	Real-time GPS coordinates broadcast to all family members via Server-Sent Events
SOS Alerts	One-tap emergency button that instantly notifies all family members via push notification + in-app alert
Geofencing	PostGIS-powered geographic zones — automatic entry/exit event logging
Family Circles	Invite-code-based family groups; child users join via parent-issued invite code (no subscription required)
Subscription Plans	Free / Family / Premium plans with Razorpay (India), Stripe (global), M-Pesa (Kenya), PayPal payment gateways
Payment-Gated Signup	Account is created only after payment is confirmed; DB is never written on failed/cancelled payments
Admin Control	Full administrative panel with user management, system monitoring, and broadcast messaging

Platform Components



2. Technology Stack

Backend

Technology	Version	Purpose
Node.js	20.x LTS	Runtime environment
Express.js	5.2.x	HTTP server framework
PostgreSQL	15+	Primary relational database
PostGIS	3.x	Geospatial extension for safe zone geometry
JWT	jsonwebtoken	Stateless authentication tokens + short-lived phone_token for signup flow
bcrypt	bcryptjs	Password hashing (also generates random hashes for OTP-only users)
Zod	3.x	Runtime request schema validation
express-rate-limit	7.x	API rate limiting
Razorpay	SDK	India payment gateway
Stripe	SDK	Global payment gateway
CORS	cors	Cross-origin resource sharing
dotenv	16.x	Environment variable management

Web Frontend

Technology	Version	Purpose
React	18.x	UI library
TypeScript	5.x	Type safety
Vite	8.x	Build tool and dev server
CSS Modules	—	Scoped component styles
Leaflet.js	1.9.x	Interactive maps (4 tile styles)
react-leaflet	4.x	React bindings for Leaflet
Fullscreen API	Native	Both ParentPanel and ChildPanel support `requestFullscreen()`

Mobile Application

Technology	Version	Purpose
React Native	0.74+	Cross-platform mobile framework
Expo	54	Development platform and managed workflow
expo-location	17.x	Device GPS access
expo-notifications	0.28.x	Push notification handling
expo-blur	13.x	iOS-style blur effects
react-native-maps	1.14.x	Native map integration (Google Maps / Apple Maps)
@react-navigation/native	6.x	Screen navigation
@react-navigation/bottom-tabs	6.x	Bottom tab navigation
react-native-safe-area-context	4.x	Safe area insets for notched devices
@expo/vector-icons	14.x	Ionicons icon set

Infrastructure

Technology	Purpose
Caddy 2	Reverse proxy with automatic TLS (Let's Encrypt)
PM2	Node.js process manager with auto-restart
Cloudflare R2	S3-compatible object storage for user avatars
MSG91	SMS OTP delivery service
Expo Push API	Mobile push notification delivery
Razorpay	Payment gateway — India (INR)
Stripe	Payment gateway — Global (USD/EUR/GBP)
M-Pesa / PesaPal	Payment gateways — East Africa (KES/UGX/TZS)
PayPal	Payment gateway — Global fallback

3. System Architecture Overview

Request Flow

Web Frontend Serving

---

4. Infrastructure & Deployment

Process Management — PM2

Three processes managed by PM2 via `ecosystem.config.js`:

Process Name	Details
gravity-api	Express backend — Port 8002 (PM2 id=58) Max memory: 500MB, autorestart: true
gravity-web	Static file server — Port 8090 (PM2 id=59) Max memory: 200MB, autorestart: true
gravity-traccar	Traccar GPS ingestion service Port 8082 (Caddy routes /telemetry/*)

VPS: 187.127.148.237 | Project path: `/var/www/gravitypro/`

Deploy commands (run on VPS):

```
cd /var/www/gravitypro && git pull origin main
cd landing-react && npm run build
pm2 restart 59     # restart web server (id=59)
pm2 restart 58     # restart API (id=58)
```

PM2 commands:

```
pm2 start ecosystem.config.js    # Start all processes
pm2 reload ecosystem.config.js   # Zero-downtime reload
pm2 logs gravity-api             # View API logs
pm2 monit                        # Real-time dashboard
pm2 restart 59                   # Restart web server by PM2 id
```

Caddy Reverse Proxy

Caddy handles TLS termination and routing at `gravitypro.kvlbusinesssolutions.com`:

```
Request → gravitypro.kvlbusinesssolutions.com
│
├── /api/v1/sse/*   → localhost:3000 (flush_interval: -1 for SSE streaming)
├── /telemetry/*    → localhost:8082 (Traccar GPS ingestion)
└── /api/*          → localhost:3000 (Express API)
```

SSE Special Config: `flush_interval -1` disables response buffering — critical for Server-Sent Events to stream in real-time without Caddy buffering event chunks.

Security Headers Applied by Caddy:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `Referrer-Policy: strict-origin-when-cross-origin`
- gzip compression on all responses

Logs

Process	Output Log	Error Log
gravity-api	/tmp/gravity-api-out.log	/tmp/gravity-api-error.log
gravity-web	/tmp/gravity-web-out.log	/tmp/gravity-web-error.log
gravity-traccar	/tmp/gravity-traccar-out.log	/tmp/gravity-traccar-error.log
Caddy	/var/log/caddy/gravity.log (JSON)	—

---

5. Backend API Architecture

App Entry Point — `backend/src/app.js`

```
app.js
│
├── Global middleware
│   ├── cors({ origin: '*' })
│   ├── express.json()
│   └── express-rate-limit (1000 requests / 15 minutes per IP)
│
├── Route mounting
│   ├── /api/v1/auth          → routes/auth.js
│   ├── /api/v1/users         → routes/users.js
│   ├── /api/v1/circles       → routes/circles.js
│   ├── /api/v1/sos           → routes/sos.js
│   ├── /api/v1/geofences     → routes/geofences.js
│   ├── /api/v1/locations     → routes/locations.js
│   ├── /api/v1/media         → routes/media.js
│   ├── /api/v1/sse           → routes/sse.js
│   ├── /api/v1/payments      → routes/payments.js
│   ├── /api/v1/subscriptions → routes/subscriptions.js
│   └── /api/v1/admin         → routes/admin.js
│
└── Global error handler
    └── Returns { error: message } JSON on uncaught route errors
```

Middleware

`authenticate` — `backend/src/middleware/auth.js`

- Reads `Authorization: Bearer <token>` header

- Verifies JWT with `JWT_SECRET` from environment
- Attaches `req.user = { id, phone, role }` to request
- Returns `401 Unauthorized` if token missing or invalid

`validate(schema)` — `backend/src/middleware/validate.js`

- Accepts a Zod schema object
- Parses `req.body` against schema
- Returns `400 Bad Request` with field-level error details on mismatch
- On success, replaces `req.body` with the parsed (typed) result

`adminAuth` — inline in `routes/admin.js`

- Reads `x-admin-token` request header
- Compares against `ADMIN_TOKEN` environment variable
- Returns `401 Unauthorized` on mismatch

Database Connection — `backend/src/config/db.js`

Uses `node-postgres` (`pg`) with connection pooling:

```
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,  // Neon PostgreSQL pooled URL
  max: 20,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
})
// All queries use parameterized statements ($1, $2...)
```

Database host: Neon PostgreSQL (cloud-managed, pooled connection via `DATABASE_URL`)

---

6. Database Schema

Tables Overview

12 tables total (3 added since initial schema):

```
┌──────────────────────────────────────────────────────────────────┐
│                     DATABASE SCHEMA                              │
│                                                                  │
│  users ──────────────────────────────────────────────────────    │
│  ├── id (UUID PK)                                                │
│  ├── phone (UNIQUE, VARCHAR 20)                                  │
│  ├── name (VARCHAR 100)                                          │
│  ├── email (UNIQUE, VARCHAR 255)                                 │
│  ├── password_hash (VARCHAR 255, NOT NULL — random hash for OTP users)│
│  ├── avatar_url (TEXT)                                           │
│  ├── push_token (TEXT — Expo push token)                         │
│  ├── country_code (VARCHAR 5, default 'IN')                      │
│  ├── account_type ('parent'|'child', default 'parent') ← migration 005│
│  ├── google_id (TEXT) ← migration 004                            │
│  ├── current_plan ('free'|'family'|'premium', default 'free') ← 006│
│  ├── is_banned (BOOLEAN)                                         │
│  └── created_at / updated_at                                     │
│                                                                  │
│  circles ────────────────────────────────────────────────────    │
│  ├── id (UUID PK)                                                │
│  ├── name (VARCHAR 100)                                          │
│  ├── invite_code (VARCHAR 12, UNIQUE)                            │
│  ├── created_by (FK → users.id)                                  │
│  └── created_at / updated_at                                     │
│                                                                  │
│  circle_members ─────────────────────────────────────────────    │
│  ├── id (UUID PK)                                                │
│  ├── circle_id (FK → circles.id ON DELETE CASCADE)               │
│  ├── user_id (FK → users.id ON DELETE CASCADE)                   │
│  ├── role ('admin'|'member', default 'member')                   │
│  ├── joined_at                                                   │
│  └── UNIQUE(circle_id, user_id)                                  │
│                                                                  │
│  phone_otps ─────────────────────────────────────────────────    │
│  ├── id (UUID PK)                                                │
│  ├── phone (VARCHAR 20)                                          │
│  ├── code (6-digit string)                                       │
│  ├── expires_at (TIMESTAMPTZ, 10 min from creation)              │
│  └── used (BOOLEAN, default false)                               │
│                                                                  │
│  device_locations (PostGIS) ─────────────────────────────    │
│  ├── id (UUID PK)                                                │
│  ├── user_id (FK → users.id ON DELETE CASCADE)                   │
│  ├── geom (GEOMETRY Point, SRID 4326)                            │
│  ├── accuracy, speed, bearing, altitude (FLOAT)                  │
│  ├── battery_level (FLOAT)                                       │
│  └── recorded_at / created_at                                    │
│                                                                  │
│  user_latest_locations (live lookup table) ──────────────    │
│  ├── user_id (UUID PK, FK → users.id ON DELETE CASCADE)          │
│  ├── geom (GEOMETRY Point, SRID 4326)                            │
│  ├── accuracy, battery_level (FLOAT)                             │
│  └── updated_at                                                  │
```

```
  |                                                                  |
  |  safe_zones (PostGIS) ──────────────────────────────────────     |
  |  ├── id (UUID PK)                                                |
  |  ├── circle_id (FK → circles.id ON DELETE CASCADE)               |
  |  ├── name (VARCHAR 100)                                          |
  |  ├── geom (GEOMETRY Polygon, SRID 4326 — ST_Buffer circle)       |
  |  ├── radius_meters (FLOAT)                                       |
  |  ├── created_by (FK → users.id)                                  |
  |  └── created_at / updated_at                                     |
  |                                                                  |
  |  geofence_events ──────────────────────────────────────────     |
  |  ├── id (UUID PK)                                                |
  |  ├── user_id (FK → users.id ON DELETE CASCADE)                   |
  |  ├── safe_zone_id (FK → safe_zones.id ON DELETE CASCADE)         |
  |  ├── event_type ('entry'|'exit')                                 |
  |  ├── geom (GEOMETRY Point, SRID 4326 — where event occurred)     |
  |  └── created_at                                                  |
  |                                                                  |
  |  sos_events ────────────────────────────────────────────────     |
  |  ├── id (UUID PK)                                                |
  |  ├── user_id (FK → users.id)                                     |
  |  ├── circle_id (FK → circles.id)                                 |
  |  ├── message (TEXT)                                              |
  |  ├── latitude / longitude (FLOAT)                                |
  |  ├── resolved (BOOLEAN, default false)                           |
  |  └── created_at                                                  |
  |                                                                  |
  |  subscription_plans ← migration 006 ──────────────────────     |
  |  ├── id (TEXT PK: 'free'|'family'|'premium')                     |
  |  ├── display_name (TEXT)                                         |
  |  ├── price_usd, price_inr, price_kes, price_eur, price_gbp       |
  |  ├── max_members, max_circles, history_days (INT)                |
  |  ├── features (JSONB array)                                      |
  |  └── is_active (BOOLEAN)                                         |
  |                                                                  |
  |  user_subscriptions ← migration 006 ──────────────────────     |
  |  ├── id (UUID PK)                                                |
  |  ├── user_id (FK → users.id ON DELETE CASCADE)                   |
  |  ├── plan_id (TEXT, default 'free')                              |
  |  ├── status ('active'|'cancelled'|'expired'|'pending')           |
  |  ├── gateway (TEXT — 'razorpay'|'stripe'|'paypal'|'mpesa'|'free')|
  |  ├── gateway_subscription_id, gateway_customer_id (TEXT)         |
  |  ├── current_period_start / current_period_end (TIMESTAMPTZ)     |
  |  └── cancelled_at / created_at / updated_at                      |
  |                                                                  |
  |  payment_orders ← migration 006 + 007 ──────────────────     |
  |  ├── id (UUID PK)                                                |
  |  ├── user_id (UUID NULLABLE FK → users.id) ← 007: made nullable  |
  |  ├── plan_id (TEXT)                                              |
  |  ├── gateway (TEXT)                                              |
  |  ├── gateway_order_id / gateway_payment_id (TEXT)                |
  |  ├── amount / currency (NUMERIC / TEXT)                          |
  |  ├── status ('pending'|'completed'|'failed'|'cancelled')         |
  |  ├── phone (TEXT) ← 007: added for pre-registration tracking     |
```

```
  │  ├── metadata (JSONB) ← 007: stores name/email during signup        │
  │  └── created_at / updated_at                                        │
  └─────────────────────────────────────────────────────────────────────┘
```

Database Migrations Applied

File	Description
`001_initial.sql`	Core schema: users, circles, circle_members, phone_otps, device_locations, user_latest_locations, safe_zones, geofence_events
`004_add_google_id.sql`	Adds `google_id` column to users for Google OAuth
`005_add_account_type.sql`	Adds `account_type` column ('parent'/'child') to users
`006_subscriptions.sql`	Adds subscription_plans, user_subscriptions, payment_orders; adds `current_plan` to users; seeds plan catalog
`007_anon_payments.sql`	Makes `payment_orders.user_id` nullable; adds `metadata` JSONB and `phone` TEXT columns to payment_orders

PostGIS Spatial Operations

Safe zones are stored as geographic buffer circles:

```
-- Creating a safe zone (200-meter radius around a point)
INSERT INTO safe_zones (circle_id, name, geom, radius_meters, created_by)
VALUES (
  $1, $2,
  ST_Buffer(
    ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography,
    $radius_meters
  )::geometry,
  $radius_meters, $user_id
)

-- Check if a point is inside any safe zone for a circle
SELECT sz.id, sz.name
FROM safe_zones sz
WHERE sz.circle_id = $circleId
  AND ST_Within(
    ST_SetSRID(ST_MakePoint($lng, $lat), 4326),
    sz.geom
  )
```

All spatial data uses **SRID 4326** (WGS84 — standard GPS coordinate system).

---

7. Database Indexes & Performance

Indexes Applied

Table	Column(s)	Index Type	Purpose
`users`	`phone`	UNIQUE B-tree	OTP lookup, login
`users`	`email`	UNIQUE B-tree	Email uniqueness, Google OAuth lookup
`circles`	`invite_code`	UNIQUE B-tree	Join-by-code lookup
`circle_members`	`(circle_id, user_id)`	Composite UNIQUE B-tree	Membership check (runs on every authenticated request)
`device_locations`	`user_id`	B-tree	Location history queries
`device_locations`	`geom`	GIST	Spatial queries
`device_locations`	`recorded_at DESC`	B-tree	Latest-first pagination
`safe_zones`	`circle_id`	B-tree	Zone list per circle
`safe_zones`	`geom`	GIST	PostGIS spatial queries (ST_Within)
`geofence_events`	`user_id`	B-tree	Event history per user
`geofence_events`	`created_at DESC`	B-tree	Latest-first pagination
`payment_orders`	`user_id`	B-tree	Order history per user
`payment_orders`	`gateway_order_id`	B-tree	Webhook lookup by gateway reference
`user_subscriptions`	`user_id`	B-tree	Active plan lookup
`phone_otps`	`(phone, used, expires_at)`	Composite B-tree	OTP verification

Query Performance Notes

- `circle_members` **check** runs on every circle-data API call. The composite UNIQUE index on `(circle_id, user_id)` makes this an O(log n) lookup.
- **PostGIS GIST index** on `safe_zones.geom` is critical — without it, geofence entry/exit detection would do a full table scan on every location update.
- `device_locations` grows unboundedly. The Admin Panel "Purge Old Locations" function deletes records older than a configurable number of days. Recommended: monthly cron.
- **Connection pool** is set to `max: 20`. At peak load, new requests queue until a connection is free.

---

8. Authentication System

Authentication Methods

```
┌──────────────────────────────────────────────────────────────┐
│                  AUTHENTICATION FLOWS                        │
│                                                              │
│  1. PHONE OTP LOGIN (Existing users)                         │
│     POST /auth/send-otp → MSG91 SMS                          │
│     POST /auth/verify-otp → DB check → JWT token             │
│     (Returns 404 if phone not yet registered)                │
│                                                              │
│  2. PHONE OTP SIGNUP (New users — see Section 9)             │
│     POST /auth/send-otp → MSG91 SMS                          │
│     POST /auth/verify-phone → phone_token JWT (30min)        │
│     → Details form (name, email, type, country)              │
│     → For parents: Plan selection → Payment                  │
│     → POST /auth/register-free (child or free parent)        │
│     → POST /auth/register-with-payment (paid parent)         │
│                                                              │
│  3. CLASSIC REGISTER (Legacy, still active)                  │
│     POST /auth/send-otp                                      │
│     POST /auth/register { phone, name, otp, account_type }   │
│                                                              │
│  4. GOOGLE OAUTH (Social)                                    │
│     POST /auth/google { id_token }                           │
│     → Decode JWT without library                             │
│     → Find or create user by google_id or email              │
│     → Return session JWT                                     │
│                                                              │
│  5. ADMIN (Separate System)                                  │
│     POST /admin/auth { password }                            │
│     → Compare against ADMIN_TOKEN env                        │
│     → Return admin_token (sent as x-admin-token header)      │
└──────────────────────────────────────────────────────────────┘
```

JWT Token Types

```
// 1. Session token — returned after login or registration
// Payload: { userId: "uuid" }
// Storage: localStorage 'gravity_token' (web) / AsyncStorage 'auth_token' (mobile)
// Expiry: JWT_EXPIRES_IN env (default '7d')

// 2. phone_token — short-lived, used ONLY during signup flow
// Payload: { phone: "+919876543210", type: "phone_verified" }
// Storage: React state only — never persisted
// Expiry: 30 minutes
// Purpose: Proves phone was verified without creating a DB entry yet
```

OTP Rate Limiting

- Maximum **3 OTP requests per phone number per 10 minutes**
- OTPs expire after **10 minutes**
- Each OTP is single-use (marked `used = TRUE` after verification)
- Old unused OTPs for the same phone are invalidated before issuing a new one
- Fallback: if `MSG91_AUTH_KEY` is not set, OTP is logged to server console and returned in API response as `dev_otp`

Password Security

- Passwords hashed with `bcrypt` (12 salt rounds)
- `bcrypt.compare()` used for all password verifications
- **OTP-only users** (most users): a random 32-byte hex string is hashed and stored to satisfy the `password_hash NOT NULL` constraint — the user has no usable password

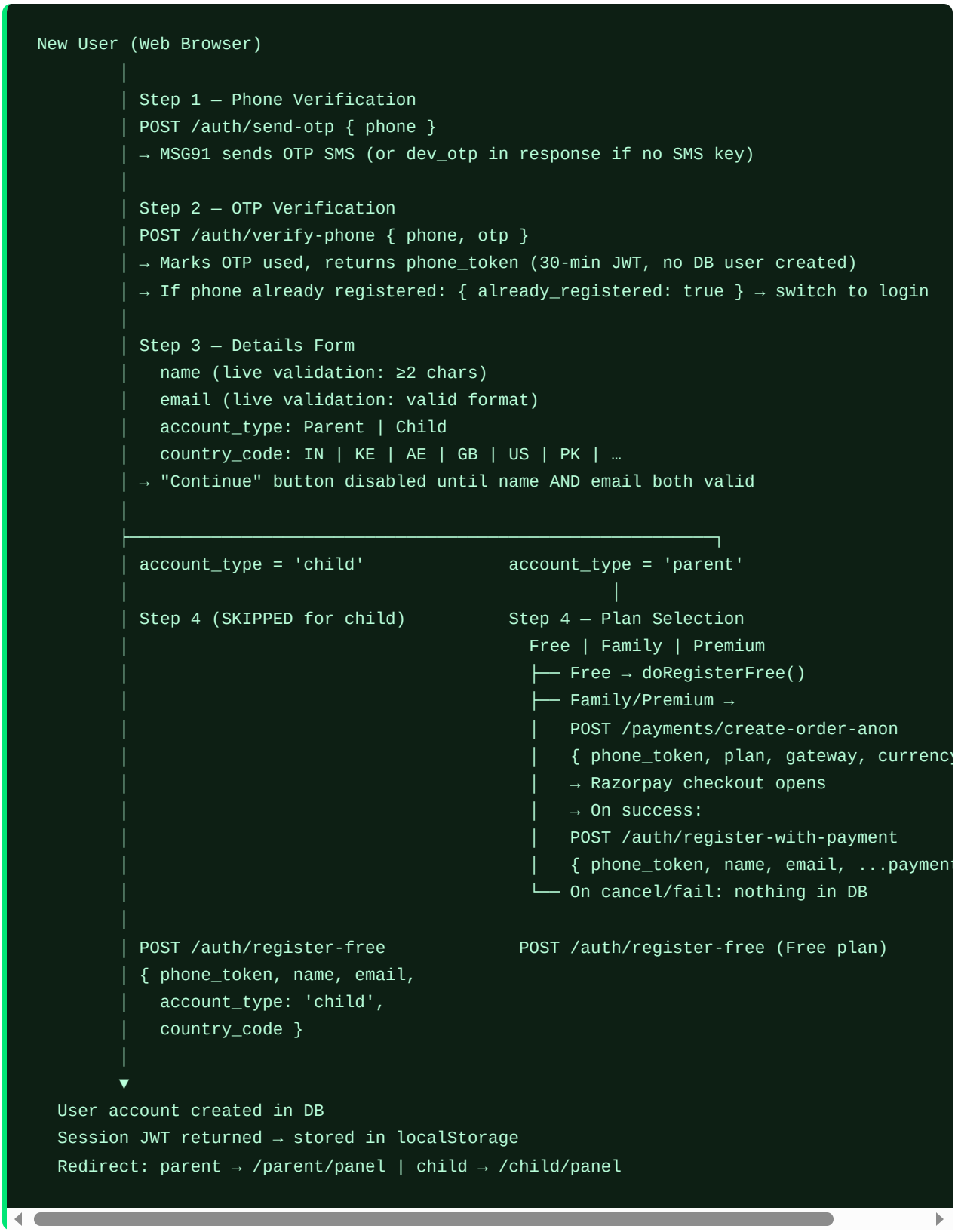
---

9. Payment-Gated Signup Flow

Design Principle

Zero database entries until payment is confirmed. If a user cancels payment or if payment fails, no user record is created.

Flow Diagram



Child Account Special Behavior

- Child accounts **do not need a subscription**
- Child accounts **skip the plan selection step entirely**

- A child joins a family circle by entering a **parent-issued invite code** from within the ChildPanel
- The invite code flow: ChildPanel → Settings → Join Circle → enter 12-character invite code

Endpoints Used in Signup Flow

Endpoint	Method	Auth	Description
`/auth/send-otp`	POST	None	Send OTP to phone
`/auth/verify-phone`	POST	None	Verify OTP → return phone_token (no DB write)
`/payments/create-order-anon`	POST	phone_token in body	Create Razorpay order before user exists
`/auth/register-free`	POST	phone_token in body	Create free/child account
`/auth/register-with-payment`	POST	phone_token in body	Create account + activate subscription atomically

---

10. Subscription & Payment System

Plans

Plan	Price (INR)	Price (USD)	Price (KES)	Members	Circles	History
Free	₹0	$0	0	4	1	1 day
Family	₹299/mo	$5.99/mo	KES 599/mo	6	3	7 days
Premium	₹499/mo	$9.99/mo	KES 999/mo	15	10	30 days

Multi-currency support: USD, INR, KES, EUR, GBP.

Payment Gateways

Gateway	Region	Currency	Method
Razorpay	India	INR	Credit card, UPI, Net banking, Wallets
Stripe	Global	USD, EUR, GBP	Credit/debit card
PayPal	Global	USD	PayPal account
M-Pesa	Kenya, Tanzania	KES, TZS	Mobile money
PesaPal	East Africa	KES, UGX, TZS	Mobile money + card

Subscription Lifecycle

```
New User (paid signup)
     │
     ├── POST /payments/create-order-anon → payment_orders row (user_id=NULL)
     ├── Razorpay payment succeeds
     ├── POST /auth/register-with-payment
     │    ├── Creates users row
     │    ├── Creates user_subscriptions row (status='active', period end = +1 month)
     │    ├── Updates payment_orders.status='completed', user_id=new user UUID
     │    └── Sets users.current_plan = plan_id
     │
Existing User (upgrade)
     ├── POST /payments/create-order (authenticate middleware required)
     ├── POST /payments/verify → activates new subscription
     │    ├── Cancels existing active subscription
     │    ├── Inserts new user_subscriptions row
     │    └── Updates users.current_plan
     │
Webhook (async confirmation)
     ├── POST /payments/webhook/razorpay
     ├── POST /payments/webhook/stripe
     ├── POST /payments/callback/mpesa
     └── POST /payments/callback/pesapal
          └── All call activateSub() internally
```

Payment Routes

Method	Path	Auth	Description
GET	`/payments/plans`	None	List all active plans with prices
GET	`/payments/gateways?currency=INR`	None	Available gateways for currency
POST	`/payments/create-order`	Bearer	Create order for logged-in user
POST	`/payments/create-order-anon`	phone_token in body	Create order before account exists
POST	`/payments/verify`	Bearer	Verify payment + activate subscription
GET	`/payments/status/:orderId`	Bearer	Poll order status (M-Pesa)
POST	`/payments/webhook/razorpay`	Webhook sig	Razorpay async webhook
POST	`/payments/webhook/stripe`	Webhook sig	Stripe async webhook
POST	`/payments/webhook/paypal`	Webhook sig	PayPal async webhook
POST	`/payments/callback/mpesa`	None	M-Pesa async callback
POST	`/payments/callback/pesapal`	None	PesaPal async callback

Subscription Routes

Method	Path	Auth	Description
GET	`/subscriptions/me`	Bearer	Get active subscription details
POST	`/subscriptions/cancel`	Bearer	Cancel active subscription
GET	`/subscriptions/history`	Bearer	Payment order history

11. Real-Time Communication (SSE)

Server-Sent Events Architecture

Gravity uses Server-Sent Events (SSE) instead of WebSockets for real-time updates. SSE is a one-directional push channel from server to client over a persistent HTTP connection.

```
┌──────────────────────────────────────────────────────────┐
│                    SSE SYSTEM                            │
│                                                          │
│  GET /api/v1/sse/stream                                  │
│  Auth: Bearer <token> OR ?token=<token> query param      │
│                                                          │
│  Connection lifecycle:                                   │
│  1. Client connects → server sets headers:               │
│     Content-Type: text/event-stream                      │
│     Cache-Control: no-cache                              │
│     Connection: keep-alive                               │
│  2. Server sends: event: connected                       │
│  3. Server registers client: clients.set(userId, res)    │
│  4. Every 25 seconds: server writes ': ping' comment     │
│     (prevents proxy/load-balancer timeout)               │
│  5. On disconnect: clients.delete(userId)                │
│                                                          │
│  Broadcasting (from any route handler):                  │
│  broadcastToCircle(circleId, { type, data })             │
│  ├── Queries circle_members for circleId                 │
│  ├── Finds each member in clients Map                    │
│  └── Writes: data: JSON.stringify(payload)\n\n           │
│                                                          │
│  Event Types Broadcast:                                  │
│  ├── location_update  — GPS coordinate from family member│
│  ├── sos_alert        — SOS triggered by family member   │
│  ├── geofence_event   — Zone entry or exit event         │
│  └── broadcast        — Admin system-wide message        │
└──────────────────────────────────────────────────────────┘
```

SSE Client Registry

```
// In-memory Map (per process):
const clients = new Map()  // userId → res (Express response object)

// Add client on connect
clients.set(userId, res)

// Remove on disconnect
req.on('close', () => clients.delete(userId))

// Broadcast to specific user
function sendToUser(userId, payload) {
  const client = clients.get(userId)
  if (client) {
    client.write(`data: ${JSON.stringify(payload)}\n\n`)
  }
}
```

Real-Time Location Update Flow

```
Mobile/Web Client
    | POST /api/v1/locations/update { lat, lng, battery }
    ▼
Express Route Handler
    | INSERT device_locations (persist to DB)
    | UPSERT user_latest_locations
    ▼
broadcastToCircle(circleId, {
    type: 'location_update',
    userId, name, lat, lng, battery, timestamp
})
    |
    ├── Parent browser (SSE open) → Map tab updates marker
    └── Other family members (SSE open) → Real-time position
```

12. File Storage — Cloudflare R2

Avatar Upload Flow (3-Step Presigned Process)

R2 Configuration

- Bucket name from `CLOUDFLARE_R2_BUCKET` environment variable
- Account ID from `CLOUDFLARE_ACCOUNT_ID`
- Access credentials: `CLOUDFLARE_R2_ACCESS_KEY_ID` + `CLOUDFLARE_R2_SECRET_ACCESS_KEY`
- Public URL prefix: `CLOUDFLARE_R2_PUBLIC_URL`
- Files are publicly readable once uploaded

---

13. Web Application Architecture

Application Entry

```
landing-react/
├── index.html          ← Vite entry point
├── server.cjs          ← Node.js static file server (Port 8090)
│                         Serves /dist, SPA fallback for all routes
└── src/
    ├── main.tsx        ← React root, ErrorBoundary, window.onerror handler
    └── pages/          ← All page components
```

Routing Structure (App.tsx)

```
/ (root)
├── /                   → Home.tsx           (Landing page — simplified)
├── /login              → Login.tsx          (Multi-step signup + login tabs)
├── /pricing            → Pricing.tsx        (Plan comparison page)
├── /checkout           → Checkout.tsx       (Checkout flow)
├── /terms              → Terms.tsx          (Terms of service)
├── /privacy            → Privacy.tsx        (Privacy policy)
├── /share              → Share.tsx          (Share/referral page)
├── /parent             → Navigate → /parent/panel
├── /parent/panel       → ParentPanel.tsx    (Auth-guarded)
├── /parent-panel       → Navigate → /parent/panel
├── /child              → Navigate → /child/panel
├── /child/panel        → ChildPanel.tsx     (Auth-guarded)
├── /child-panel        → Navigate → /child/panel
├── /admin              → Navigate → /admin/login
├── /admin/login        → AdminLogin.tsx
├── /admin/panel        → AdminPanel.tsx     (Admin auth-guarded)
└── *                   → NotFound.tsx
```

Removed pages (deleted from codebase): `Parent.tsx`, `Parent.module.css`, `Child.tsx`, `Child.module.css`

Login.tsx — Multi-Step Signup & Login

```
Login.tsx (tab switcher)
│
├── Tab: "Sign In"
│   └── phone → OTP → POST /auth/verify-otp → redirect by account_type
│
└── Tab: "Create Account" (4 steps)
    ├── Step 1: Phone input → POST /auth/send-otp
    │   └── dev_otp auto-fills OTP boxes + yellow banner shown if no SMS
    │
    ├── Step 2: OTP entry → POST /auth/verify-phone
    │   └── If already_registered → switch to Sign In tab
    │
    ├── Step 3: Details form
    │   ├── Name (live: ✓ green / ✗ red, min 2 chars)
    │   ├── Email (live: ✓ green / ✗ red, format validated)
    │   ├── Account type toggle: Parent | Child
    │   └── Country dropdown: IN | KE | AE | GB | US | PK…
    │       "Continue" button DISABLED until name AND email valid
    │       For child: button says "Create Account →"
    │       For parent: button says "Continue to Plan →"
    │
    ├── Step 4 (Parent only — child skips directly to register-free)
    │   ├── Free plan card → POST /auth/register-free
    │   ├── Family plan card → Razorpay → POST /auth/register-with-payment
    │   └── Premium plan card → Razorpay → POST /auth/register-with-payment
    │
    └── On success → localStorage.setItem('gravity_token', token)
                   → redirect to /parent/panel or /child/panel
```

Parent Panel — Architecture

```
ParentPanel.tsx
├── State
│   ├── activeTab: 'map' | 'family' | 'alerts' | 'geofence' | 'settings'
│   ├── members[]          — circle members with live location data
│   ├── sosEvents[]        — real-time SOS alert list
│   ├── geofenceEvents[]   — entry/exit event list
│   ├── safeZones[]        — all zones for this circle
│   ├── allCircles[]       — multi-circle support (switcher UI)
│   ├── circleId           — active circle UUID
│   ├── isFullscreen       — fullscreen toggle state
│   ├── showHistory        — location history overlay
│   └── historyPoints[]    — GPS trail points (last 50)
│
├── Layout (simplified — NO phone frame)
│   ├── Header: GRAVITY logo | fullscreen button (green) | logout button (red)
│   ├── Content area (full-width, min-height: 100vh)
│   └── Bottom tab bar: Map | Family | Alerts | Geofence | Settings
│
├── Fullscreen
│   ├── toggleFullscreen() → document.documentElement.requestFullscreen()
│   │                        / document.exitFullscreen()
│   ├── Listens: document.addEventListener('fullscreenchange', ...)
│   └── SVG expand/compress icon toggles based on isFullscreen state
│
├── Real-time (SSE)
│   └── EventSource('/api/v1/sse/stream?token=...')
│       ├── location_update → update member in state
│       ├── sos_alert       → prepend to sosEvents
│       └── geofence_event  → prepend to geofenceEvents
│
├── Map Tab
│   ├── Leaflet MapContainer
│   ├── 4 tile layer options: Dark (CartoDB) / Light / Satellite / Street
│   ├── Family member markers (custom avatar circles)
│   ├── Safe zone polygons (green circles with labels)
│   ├── Location history polyline (dashed green trail)
│   └── Multi-circle switcher tabs (shown if >1 circle)
│
├── Family Tab → member cards with battery, status, last-seen
│   └── Uses .reveal { opacity: 1 } — always visible (not animated)
├── Alerts Tab → SOS + geofence events feed with real-time updates
├── Geofence Tab → safe zone list + create/edit/delete modals
└── Settings Tab → avatar upload (R2), name edit, subscription info, logout
```

Child Panel — Architecture

```
ChildPanel.tsx
├── State
│   ├── activeTab: 'home' | 'map' | 'sos' | 'family' | 'alerts' | 'profile'
│   ├── todayDistance, todaySafeZones, todayCheckins — stats from /users/me/stats
│   └── isFullscreen — fullscreen toggle state
│
├── Layout (simplified — NO phone frame)
│   ├── Header: GRAVITY logo | fullscreen button (green) | logout button (red)
│   ├── Content area (full-width, min-height: 100vh)
│   └── Bottom tab bar: Home | Map | SOS | Family | Alerts | Profile
│
├── Stats Keys (fixed from backend)
│   ├── distance  (was: distance_km)
│   ├── safeZones (was: safe_zones_visited)
│   └── checkins  (was: family_checkins)
│   Fallback: s.distance ?? s.distance_km ?? 0
│
├── Fullscreen — same implementation as ParentPanel
│
├── Home Tab → stats overview (distance, safe zones, check-ins)
├── Map Tab  → Leaflet map, own location + family positions
├── SOS Tab  → Large red SOS button → POST /sos/trigger → SSE broadcast
├── Family Tab → circle members list
├── Alerts Tab → All / Geofence / SOS filter chips
└── Profile Tab → avatar upload, name/email/phone display, edit, logout
```

Admin Panel — Architecture

```
AdminPanel.tsx (1530+ lines)
├── Dual View Mode System
│   ├── Desktop Mode (default): 240px sidebar + full-width content
│   └── Mobile Mode: max 393px + bottom nav
│   └── Toggled via localStorage key 'admin_view_mode'
│
├── 7 Tabs
│   ├── Dashboard    → 9-stat grid + recent SOS (shimmer loading)
│   ├── Users        → search + role/status filter + ban/unban/delete
│   ├── Circles      → search + member count + invite regenerate + delete
│   │                  + Create Circle FAB (slide-up bottom sheet modal)
│   ├── SOS          → filter (All/New/Resolved) + bulk resolve
│   ├── Logs         → sub-tabs: Geofence events | OTP logs
│   ├── System       → DB stats, server info, SSE count, danger zone
│   └── Broadcast    → type selector + textarea + send + history
│
└── Desktop Sidebar
    ├── GRAVITY logo + tagline
    ├── 7 nav items with active highlight
    ├── SOS badge (unresolved count)
    ├── Mobile View toggle button
    └── Sign Out button
```

Home.tsx — Landing Page (Simplified)

The landing page was simplified from ~1446 lines to ~868 lines. Removed sections:

- Testimonials
- Stats bar
- How-it-works section
- Screenshots section
- Countries/global reach section
- Download app section
- Demo family characters: Pinky and Grand Mom removed from hero strip, map markers, and member list

Remaining sections: Navbar + Hero (with family avatar strip and live map demo) + 3 Feature cards + 3 Pricing cards + Footer.

CSS Design System

```
/* Core Color Palette */
--bg-primary:    #050C08   /* Deep black-green background */
--bg-secondary:  #0A1510   /* Slightly lighter surface */
--bg-glass:      rgba(10, 92, 53, 0.08)  /* Glass card effect */
--accent:        #00E676   /* Bright green — primary action */
--accent-dark:   #0A5C35   /* Dark green — secondary accent */
--danger:        #FF1744   /* SOS / error / danger red */
--text-primary:  #FFFFFF   /* Primary text */
--text-muted:    #7A9E8A   /* Secondary text */
--border:        rgba(0, 230, 118, 0.12)  /* Subtle green border */

/* CSS Module keyframes */
@keyframes fadeIn    { 0% → opacity 0 | 100% → opacity 1 }
@keyframes shimmer   { shimmer loading skeleton effect }
@keyframes sosPulse  { red pulsing ring for SOS alerts }
@keyframes slideUp   { panel slide-in from bottom }
@keyframes ping      { expanding circle pulse }
@keyframes toastIn   { notification slide-in }
```

14. Mobile Application Architecture

App Structure

```
mobile/
├── App.js                          ← Root component, NavigationContainer
├── src/
│   ├── navigation/
│   │   ├── TabNavigator.jsx        ← Bottom tabs (blur background, iOS/Android)
│   │   └── RootNavigator.jsx       ← Auth guard (Login vs Tabs)
│   ├── screens/
│   │   ├── AuthScreen.jsx          ← Phone + OTP login
│   │   ├── MapScreen.jsx           ← Live family map (React Native Maps)
│   │   ├── CirclesScreen.jsx       ← Circle management
│   │   ├── SafeZonesScreen.jsx     ← Geofence zone viewer
│   │   ├── AlertsScreen.jsx        ← SOS + geofence alerts
│   │   └── ProfileScreen.jsx       ← User profile + settings
│   ├── services/
│   │   ├── api.js                  ← Full API client (all endpoints)
│   │   ├── location.js             ← Background GPS tracking
│   │   ├── notifications.js        ← Expo push notification setup
│   │   └── offlineQueue.js         ← Queue for poor connectivity
│   ├── components/
│   │   └── (reusable UI components)
│   └── theme/
│       └── colors.js               ← Color constants (matches web theme)
```

Bottom Tab Navigation

```
Tab Bar (5 Tabs)
├── Map      → MapScreen.jsx
├── Circles  → CirclesScreen.jsx
├── Zones    → SafeZonesScreen.jsx
├── Alerts   → AlertsScreen.jsx
└── Profile  → ProfileScreen.jsx

Tab Bar Style:
- Position: absolute (floats over content)
- Background: iOS → BlurView | Android → semi-opaque dark
- Active icon: Ionicons solid + green tint
- Inactive icon: Ionicons outline + muted gray
```

MapScreen Architecture

```
MapScreen.jsx (781 lines)
├── State
│   ├── ownLocation      — device GPS (expo-location)
│   ├── familyMembers[]  — polled every 10 seconds from API
│   ├── selectedMember   — tapped marker → info card
│   └── sosVisible       — SOS confirm modal
│
├── Location Tracking
│   ├── expo-location.watchPositionAsync (HIGH_ACCURACY)
│   ├── Updates own marker every position change
│   └── POST /api/v1/locations/update every 30 seconds
│
├── Map Display
│   ├── react-native-maps MapView
│   ├── PROVIDER_GOOGLE on Android, default on iOS
│   ├── DARK_MAP_STYLE (custom JSON style array)
│   ├── Own location: green animated pulsing dot (Animated.loop)
│   └── Family markers: circular avatar with initials + name label
│
└── SOS Trigger
    POST /api/v1/sos/trigger { circleId, message, lat, lng }
    → Server SSE broadcast to all members
    → Server sends Expo push notifications
```

Service Layer

`api.js` — API Client

```
const BASE_URL = process.env.EXPO_PUBLIC_API_URL || 'https://gravitypro.kvlbusinesssoluti
const getToken = () => AsyncStorage.getItem('auth_token')
// Methods: auth, users, circles, sos, geofences, locations, media, subscriptions
```

`location.js` — Background Location

```
// Update interval: 30 seconds
// Accuracy: LocationAccuracy.High
// Distance filter: 10 meters (only sends if moved >10m)
// Sends: { latitude, longitude, accuracy, battery_level }
```

`notifications.js` — Push Setup

```
// On login: request permissions → get Expo push token
// POST /api/v1/users/me/push-token { push_token }
// Token stored in users.push_token column
```

`offlineQueue.js` — Connectivity Resilience

```
// Queue location updates when offline
// Retry on reconnect (NetInfo listener)
// Max queue size: 100 entries
```

---

15. Mobile App Permissions

Android Permissions

Permission	Purpose
`ACCESS_FINE_LOCATION`	GPS tracking (high accuracy)
`ACCESS_COARSE_LOCATION`	Approximate location fallback
`ACCESS_BACKGROUND_LOCATION`	Location updates when app backgrounded
`RECEIVE_BOOT_COMPLETED`	Restart location service after reboot
`VIBRATE`	Vibrate on SOS alert received
`POST_NOTIFICATIONS`	Show push notifications (Android 13+)

iOS Permissions

Key	Purpose
`NSLocationWhenInUseUsageDescription`	Map screen location sharing
`NSLocationAlwaysAndWhenInUseUsageDescription`	Background tracking
`NSLocationAlwaysUsageDescription`	iOS 10 compatibility

16. Push Notifications

Expo Push Notification Flow

```
Family Member (Mobile)
    | POST /api/v1/sos/trigger { circleId, message, lat, lng }
    ▼
Express SOS Route Handler
    | 1. INSERT INTO sos_events
    | 2. broadcastToCircle() → SSE to all open connections
    | 3. SELECT push_token FROM users WHERE id IN (circle members)
    | 4. Filter tokens starting with 'ExponentPushToken['
    ▼
Expo Push API (https://exp.host/--/api/v2/push/send)
    | Chunked: max 100 tokens per request
    | { to, title: '🚨 SOS Alert', body, data: { type, circleId } }
    ▼
Apple APNs / Google FCM → Device notification
```

---

17. Geofencing System

Zone Creation

```
INSERT INTO safe_zones (circle_id, name, geom, radius_meters, created_by)
VALUES ($circle_id, $name,
  ST_Buffer(
    ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography,
    $radius_meters
  )::geometry,
  $radius_meters, $user_id
)
```

Entry/Exit Detection

When a device posts a location update:

1. Backend checks if the new position intersects any safe zones in the user's circle (`ST_Within`)
2. Compares with previous position's zone membership
3. If zone membership changed → INSERT into `geofence_events`

4. Broadcast geofence event via SSE to all circle members

Geofence API

Endpoint	Method	Description
`/api/v1/geofences/circle/:circleId`	GET	List all safe zones
`/api/v1/geofences/events/:circleId`	GET	Entry/exit event log
`/api/v1/geofences/`	POST	Create new safe zone
`/api/v1/geofences/:id`	PATCH	Edit zone name/position/radius
`/api/v1/geofences/:id`	DELETE	Delete zone

18. Security Model

Defense Layers

```
Layer 1: Transport Security
├── TLS 1.3 via Caddy (automatic Let's Encrypt)
└── HSTS enforced by Caddy

Layer 2: Network Controls
├── Security headers (X-Frame-Options: DENY, etc.)
├── Rate limiting: 1000 requests / 15 minutes per IP
├── OTP rate limit: 3 OTPs / 10 minutes per phone
└── CORS: Configured per environment

Layer 3: Authentication
├── Session JWT with configurable expiry (default 7 days)
├── phone_token JWT (30-min, type-checked, signup use only)
├── JWT_SECRET from environment (never hardcoded)
├── Separate admin token (x-admin-token header)
├── bcrypt password hashing (12 rounds)
└── Google OAuth: id_token decoded without external library

Layer 4: Authorization
├── All user routes require authenticate() middleware
├── Circle data access: circle_members checked on every query
├── Admin routes: separate x-admin-token header check
└── Geofence/zone mutations: role checked (admin vs member)

Layer 5: Data Validation
├── Zod schemas on all POST/PATCH request bodies
├── UUID format validated for all ID parameters
├── Parameterized SQL queries (no string interpolation)
└── Number range checks (radius: 50–50,000 meters)

Layer 6: Payment Security
├── phone_token type must equal 'phone_verified' for anon orders
├── Razorpay HMAC-SHA256 signature verified before account creation
├── payment_orders.user_id stays NULL until payment confirmed
└── Zero DB entries on failed/cancelled payments

Layer 7: Storage
├── Passwords: bcrypt hashed, never logged
├── OTPs: single-use, time-limited
├── Avatar files: uploaded directly to R2 (never through API server)
└── Push tokens: stored per-user, not exposed in API responses
```

19. API Endpoint Reference

Base URL: `https://gravitypro.kvlbusinesssolutions.com/api/v1`
Auth: All user endpoints require `Authorization: Bearer <jwt_token>` header
Admin Auth: Admin endpoints require `x-admin-token: <admin_token>` header

Authentication — `/auth`

Method	Path	Auth	Description
POST	`/auth/send-otp`	None	Send OTP to phone (MSG91)
POST	`/auth/verify-otp`	None	Verify OTP → return JWT (login for existing users)
POST	`/auth/verify-phone`	None	Verify OTP → return phone_token (signup step 2)
POST	`/auth/register`	None	Classic register with OTP
POST	`/auth/register-free`	phone_token in body	Create free/child account post-OTP verification
POST	`/auth/register-with-payment`	phone_token in body	Create account + activate subscription atomically
POST	`/auth/google`	None	Login/register with Google id_token

Users — `/users`

Method	Path	Auth	Description
GET	`/users/me`	Bearer	Get own profile
PATCH	`/users/me`	Bearer	Update name / email / push_token
POST	`/users/me/location`	Bearer	Post current GPS location + SSE broadcast
POST	`/users/me/battery`	Bearer	Post battery level
GET	`/users/me/stats`	Bearer	Get today's stats { distance, safeZones, checkins }
GET	`/users/me/location-history`	Bearer	Get last N location points

Circles — `/circles`

Method	Path	Auth	Description
POST	`/circles`	Bearer	Create new circle
POST	`/circles/join`	Bearer	Join circle with invite code
GET	`/circles/my`	Bearer	Get all circles user belongs to
GET	`/circles/:id/members`	Bearer	Get circle member list
DELETE	`/circles/:id/members/:userId`	Bearer	Remove member from circle

SOS — `/sos`

Method	Path	Auth	Description
POST	`/sos/trigger`	Bearer	Trigger SOS alert + SSE broadcast + push
GET	`/sos/history`	Bearer	Get SOS event history for circle
PATCH	`/sos/:id/resolve`	Bearer	Mark SOS as resolved

Geofences — `/geofences`

Method	Path	Auth	Description
GET	`/geofences/circle/:circleId`	Bearer	List safe zones
GET	`/geofences/events/:circleId`	Bearer	Entry/exit event log
POST	`/geofences`	Bearer	Create new safe zone
PATCH	`/geofences/:id`	Bearer	Edit safe zone
DELETE	`/geofences/:id`	Bearer	Delete safe zone

Locations — `/locations`

Method	Path	Auth	Description
POST	`/locations/update`	Bearer	Post location + SSE broadcast
GET	`/locations/circle/:circleId`	Bearer	Get latest location for all members

Media — `/media`

Method	Path	Auth	Description
GET	`/media/avatar-upload-url`	Bearer	Get R2 presigned PUT URL

SSE — `/sse`

Method	Path	Auth	Description
GET	`/sse/stream`	Bearer OR `?token=`	Open SSE stream

Payments — `/payments`

Method	Path	Auth	Description
GET	`/payments/plans`	None	List active plans with multi-currency prices
GET	`/payments/gateways`	None	Available gateways for currency
POST	`/payments/create-order`	Bearer	Create order for logged-in user
POST	`/payments/create-order-anon`	phone_token in body	Create order before account exists
POST	`/payments/verify`	Bearer	Verify payment + activate subscription
GET	`/payments/status/:orderId`	Bearer	Poll order status (M-Pesa)
POST	`/payments/webhook/razorpay`	Sig	Razorpay webhook
POST	`/payments/webhook/stripe`	Sig	Stripe webhook
POST	`/payments/webhook/paypal`	None	PayPal webhook
POST	`/payments/callback/mpesa`	None	M-Pesa callback
POST	`/payments/callback/pesapal`	None	PesaPal callback

Subscriptions — `/subscriptions`

Method	Path	Auth	Description
GET	`/subscriptions/me`	Bearer	Active subscription with plan details
POST	`/subscriptions/cancel`	Bearer	Cancel active subscription
GET	`/subscriptions/history`	Bearer	Payment order history (last 20)

Admin — `/admin`

Method	Path	Admin Auth	Description
POST	`/admin/auth`	None	Admin login → returns admin_token
GET	`/admin/dashboard`	x-admin-token	9 platform statistics
GET	`/admin/users`	x-admin-token	All users with filters
PATCH	`/admin/users/:id/ban`	x-admin-token	Ban/unban user
DELETE	`/admin/users/:id`	x-admin-token	Delete user
GET	`/admin/circles`	x-admin-token	All circles with stats
POST	`/admin/circles`	x-admin-token	Create circle by owner phone
PATCH	`/admin/circles/:id/invite-code`	x-admin-token	Regenerate invite code
DELETE	`/admin/circles/:id`	x-admin-token	Delete circle
GET	`/admin/sos`	x-admin-token	All SOS events with filters
PATCH	`/admin/sos/:id/resolve`	x-admin-token	Resolve SOS event
GET	`/admin/geofence-events`	x-admin-token	All geofence events
GET	`/admin/otps`	x-admin-token	OTP log with phone search
GET	`/admin/system`	x-admin-token	DB size, uptime, SSE count

Method	Path	Admin Auth	Description
POST	`/admin/broadcast`	x-admin-token	Broadcast SSE message to all
DELETE	`/admin/purge-locations`	x-admin-token	Delete old location data

20. API Error Response Format

All error responses follow a consistent JSON structure:

```
{ "error": "Human-readable error message" }
```

HTTP Status Codes

Code	Meaning	When Returned
`200 OK`	Success	GET / PATCH / DELETE success
`201 Created`	Resource created	POST success
`400 Bad Request`	Validation failed	Zod mismatch, invalid UUID, missing field
`401 Unauthorized`	Auth required	Missing token, expired token, invalid phone_token
`403 Forbidden`	Access denied	User not a circle member, insufficient role
`404 Not Found`	Resource missing	User / circle / zone ID not found
`409 Conflict`	Duplicate	Phone already registered
`429 Too Many Requests`	Rate limit	>1000 req/15min or >3 OTPs/10min
`500 Internal Server Error`	Server error	Unhandled DB errors

21. API Request & Response Examples

Send OTP

```
POST /api/v1/auth/send-otp
Content-Type: application/json

{ "phone": "+919876543210" }
```

```
{ "success": true, "sms_sent": true }
```

```
{ "success": true, "sms_sent": false, "dev_otp": "847291" }
```

Verify Phone (Signup Step 2)

```
POST /api/v1/auth/verify-phone
Content-Type: application/json

{ "phone": "+919876543210", "otp": "847291" }
```

```
{ "verified": true, "phone_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

Create Anon Payment Order

```
POST /api/v1/payments/create-order-anon
Content-Type: application/json

{
  "phone_token": "eyJ...",
  "plan": "family",
  "gateway": "razorpay",
  "currency": "INR",
  "name": "Rahul Sharma",
  "email": "rahul@gmail.com"
}
```

```
{
  "success": true,
  "orderId": "uuid",
  "gatewayOrderId": "order_xxx",
  "clientData": { "key": "rzp_test_xxx", "amount": 29900, "currency": "INR" }
}
```

Register With Payment

```
POST /api/v1/auth/register-with-payment
Content-Type: application/json

{
  "phone_token": "eyJ...",
  "name": "Rahul Sharma",
  "email": "rahul@gmail.com",
  "account_type": "parent",
  "country_code": "IN",
  "plan": "family",
  "gateway": "razorpay",
  "gatewayOrderId": "order_xxx",
  "gatewayPaymentId": "pay_xxx",
  "signature": "hmac_sha256_signature"
}
```

```
{
  "user": { "id": "uuid", "name": "Rahul", "account_type": "parent", "current_plan": "fami
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Register Free (Child)

```
POST /api/v1/auth/register-free
Content-Type: application/json

{
  "phone_token": "eyJ...",
  "name": "Ananya",
  "email": "ananya@gmail.com",
  "account_type": "child",
  "country_code": "IN"
}
```

```
{
  "user": { "id": "uuid", "name": "Ananya", "account_type": "child", "current_plan": "fre
  "token": "eyJ..."
}
```

SSE Stream Event Payloads

```
data: {"type":"location_update","userId":"a3f9...","name":"Rahul","latitude":28.6139,"lon

data: {"type":"sos_alert","userId":"b7e2...","name":"Ananya","latitude":28.55,"longitude"

data: {"type":"geofence_event","userId":"a3f9...","name":"Rahul","eventType":"exit","zone
```

---

22. Data Flow Diagrams

Payment-Gated Signup Flow

```
Browser
  | Step 1: POST /auth/send-otp → OTP SMS
  |
  | Step 2: POST /auth/verify-phone → phone_token (30min JWT)
  |         ↳ No DB entry yet
  |
  | Step 3: User fills name/email/type/country
  |
  | Step 4: (parent + paid plan)
  |   POST /payments/create-order-anon { phone_token, plan, gateway }
  |   → payment_orders row: user_id=NULL, status='pending'
  |   → Razorpay checkout opens in browser
  |
  |   [User pays successfully]
  |
  | Step 5: POST /auth/register-with-payment { phone_token, ...paymentData }
  |   ├── Verify Razorpay HMAC-SHA256 signature
  |   ├── INSERT users (with random bcrypt hash for password_hash)
  |   ├── INSERT user_subscriptions (status='active', +1 month period)
  |   ├── UPDATE payment_orders SET status='completed', user_id=newUserId
  |   └── UPDATE users SET current_plan=planId
  |   → Return { user, token }
  |
  └── Redirect to /parent/panel
```

SOS Alert Complete Flow

```
Child Device (Mobile)
        | POST /api/v1/sos/trigger { circleId, message, latitude, longitude }
        ▼
Express SOS Route
        ├── 1. INSERT sos_events → DB
        ├── 2. broadcastToCircle(circleId, { type: 'sos_alert', ... })
        |   └── SSE clients Map → write to all open connections
        |       ├── Parent browser → Alert tab flashes red
        |       └── Admin panel → Dashboard SOS count ++
        └── 3. POST https://exp.host/--/api/v2/push/send
            └── Apple APNs + Google FCM delivery
```

Live Location Update Flow

```
User Device
        | POST /api/v1/locations/update { latitude, longitude, battery_level }
        ▼
Express Locations / Users Route
        ├── INSERT device_locations (full history)
        ├── UPSERT user_latest_locations
        ├── checkGeofenceStatus() → if zone changed → INSERT geofence_events
        └── broadcastToCircle() → location_update SSE event
            ├── Parent web (SSE open) → Leaflet marker moves
            └── Other mobile (SSE open) → Map marker moves
```

---

23. Third-Party Service Dependencies

Service	Used For	Fallback if Unavailable
MSG91	SMS OTP delivery	If key not set, OTP logged to console and returned as `dev_otp` in response
Razorpay	India payments (INR)	If not configured, create-order returns 503 with setup instructions
Stripe	Global payments (USD/EUR/GBP)	Same as Razorpay
M-Pesa / PesaPal	East Africa payments (KES)	Same as Razorpay
Cloudflare R2	Avatar image storage	If R2 down, avatar upload fails; other features continue
Expo Push API	Mobile push notifications	If unreachable, SOS push silently skipped; SSE alert still works
Apple APNs / Google FCM	iOS/Android push delivery	Managed by Expo — no direct dependency
Traccar	GPS device telemetry ingestion	Optional; core tracking via direct API still works
Let's Encrypt (via Caddy)	TLS certificates	Caddy auto-renews 30 days before expiry
Neon PostgreSQL	Cloud database	If Neon unreachable, all API requests fail; no local fallback
CartoDB tile server	Web map dark mode tiles	User can switch to Street/Satellite tile in UI

24. Scalability & Known Limitations

Current Architecture Limits

Limitation	Detail	How to Scale
SSE in-memory Map	`clients` Map lives in single Node.js process. Horizontal scaling breaks SSE.	Add Redis Pub/Sub as shared broadcast channel
Single PM2 instance	`instances: 1` in ecosystem.config.js	Change to `instances: 'max'` + Redis for SSE
`device_locations` growth	At 1 location/30s per user, 1000 active users = ~2.9M rows/day	Monthly purge via admin panel; or `pg_partman`
Connection pool cap	`max: 20` DB connections	Increase `max`; add PgBouncer for 1000+ users
No WebSocket	SSE is one-directional only	Acceptable; migrate to WebSocket if sub-second bidirectional needed
OTP via MSG91 only	Single SMS provider; outage = no OTP delivery	Add Twilio or Fast2SMS as fallback
Sync push notifications	Push sent inline in SOS handler; slow Expo API adds SOS response latency	Move to Bull + Redis background job queue
Neon DB	Cloud-managed; adds ~5ms latency vs local Postgres	Acceptable at current scale

Expected Capacity (Single Server)

Metric	Estimate
Concurrent SSE connections	~500
API requests/second	~200 (within rate limit; server handles far more)
Active users per circle	Practical limit ~50 before SSE broadcast slows
Safe zones per circle	No limit; PostGIS handles thousands efficiently

25. Monitoring & Health Checks

Health Check Endpoint

`GET /api/v1/admin/system` returns:

```
{
  "database": { "size_mb": 45.2, "tables": { "users": 312, "device_locations": 891230 } },
  "server": { "uptime_seconds": 1209600, "node_version": "v20.20.0", "memory_usage_mb": 1
  "sse": { "active_connections": 23 }
}
```

PM2 Commands

```
pm2 status              # View all process statuses
pm2 monit               # Real-time CPU + memory dashboard
pm2 logs gravity-api    # Tail API logs
pm2 logs --lines 200    # Last 200 lines all processes
```

Recommended External Monitoring (Not Yet Implemented)

Tool	Purpose
UptimeRobot (free)	HTTP uptime check every 5 minutes
PM2 Plus	Cloud dashboard for PM2 metrics
Sentry	Error tracking for Express unhandled exceptions
Grafana + Prometheus	Metrics dashboards for DB query times, SSE count

---

26. Backup & Disaster Recovery

Database Backup

Current Setup: Neon PostgreSQL handles managed backups. Manual backup recommendation:

```
# Full dump
pg_dump $DATABASE_URL -F c > /backups/gravity_$(date +%Y%m%d).dump

# Restore
pg_restore -d $DATABASE_URL /backups/gravity_20260620.dump
```

Recommended backup schedule: Daily at 2 AM, 7 daily + 4 weekly + 3 monthly retention.

Disaster Recovery Steps

```
1. Provision new Ubuntu server
2. Install Node.js 20, PM2, Caddy
3. Clone: git clone https://github.com/kamaralam1984/gravitypro /var/www/gravitypro
4. Restore DB from dump (Neon handles this automatically)
5. Copy backend/.env from secure storage
6. cd landing-react && npm run build
7. pm2 start ecosystem.config.js
8. caddy start --config caddy/Caddyfile
9. Verify: curl https://gravitypro.kvlbusinesssolutions.com/api/v1/admin/system
```

Estimated recovery time: **30–60 minutes** with prepared backups.

---

27. Data Privacy & Compliance

Data Collected

Data Type	Stored In	Sensitivity
Phone number	`users.phone`	High — personal identifier
Name / Email	`users.name`, `users.email`	High
GPS coordinates (history)	`device_locations`	Very High — movement patterns
GPS coordinates (latest)	`user_latest_locations`	Very High
Profile photo	Cloudflare R2	Medium
SOS messages	`sos_events.message`	High
Payment records	`payment_orders`	High
Expo push token	`users.push_token`	Medium

Data Isolation

- All location, SOS, and geofence data is scoped to **circles**
- A user can only query data for circles they are a member of
- Child users can only see their own circle's data
- Admins can see all data across all circles

User Data Deletion

Admin `DELETE /api/v1/admin/users/:id`:

- Removes `users` row (cascades to `circle_members`, `user_subscriptions`)
- Does **not** automatically delete `device_locations` orphaned rows
- Does **not** delete avatar from R2

GDPR Notes

If operating in EU/UK: add "Download My Data" and "Delete My Account" features; add Privacy Policy page; obtain explicit consent for location tracking on registration.

---

28. Target Markets

Region	Countries	Payment Gateway	Notes
South Asia	India	Razorpay (INR)	Primary market; MSG91 OTP works well
East Africa	Kenya	M-Pesa, PesaPal (KES)	Supported
East Africa	Uganda, Tanzania	PesaPal, M-Pesa (UGX/TZS)	Supported
Middle East	UAE	Stripe (USD)	Supported
Europe	UK	Stripe (GBP/EUR)	GDPR compliance may be required
North America	USA	Stripe, PayPal (USD)	Supported

Language: English only. No i18n implemented.

---

29. Development Setup Guide

Prerequisites

- Node.js 20.x
- PostgreSQL 15 with PostGIS extension (or Neon account)
- Git

Backend Setup

```
git clone https://github.com/kamaralam1984/gravitypro
cd Gravity/backend
npm install
cp .env.example .env   # Fill all required variables
# Run migrations
psql $DATABASE_URL -f migrations/001_initial.sql
psql $DATABASE_URL -f migrations/004_add_google_id.sql
psql $DATABASE_URL -f migrations/005_add_account_type.sql
psql $DATABASE_URL -f migrations/006_subscriptions.sql
psql $DATABASE_URL -f migrations/007_anon_payments.sql
npm run dev
```

Frontend Setup

```
cd Gravity/landing-react
npm install
npm run dev   # Vite dev server on port 5173
```

Mobile App Setup

```
cd Gravity/mobile
npm install
echo "EXPO_PUBLIC_API_URL=http://localhost:8002" > .env
npx expo start
```

Building for Production

```
# Build web frontend
cd landing-react && npm run build

# Start with PM2
pm2 start ecosystem.config.js

# Build mobile
cd mobile && npx eas build --platform android
```

---

30. Environment Configuration

Backend — `backend/.env`

```
# Database (Neon PostgreSQL)
DATABASE_URL=postgresql://neondb_owner:***@ep-odd-queen-***.neon.tech/neondb?sslmode=requ

# JWT
JWT_SECRET=<64-char-random-secret>
JWT_EXPIRES_IN=7d

# Admin
ADMIN_TOKEN=<secure-admin-password>

# SMS OTP
MSG91_AUTH_KEY=<msg91-auth-key>
MSG91_TEMPLATE_ID=<template-id>

# Cloudflare R2
CLOUDFLARE_ACCOUNT_ID=<account-id>
CLOUDFLARE_R2_ACCESS_KEY_ID=<access-key>
CLOUDFLARE_R2_SECRET_ACCESS_KEY=<secret-key>
CLOUDFLARE_R2_BUCKET=gravity-avatars
CLOUDFLARE_R2_PUBLIC_URL=https://pub-<hash>.r2.dev

# Payment Gateways
RAZORPAY_KEY_ID=rzp_live_xxx
RAZORPAY_KEY_SECRET=xxx
STRIPE_SECRET_KEY=sk_live_xxx
STRIPE_WEBHOOK_SECRET=whsec_xxx
PAYPAL_CLIENT_ID=xxx
PAYPAL_CLIENT_SECRET=xxx
MPESA_CONSUMER_KEY=xxx
MPESA_CONSUMER_SECRET=xxx
MPESA_SHORTCODE=xxx
MPESA_PASSKEY=xxx
MPESA_CALLBACK_URL=https://gravitypro.kvlbusinesssolutions.com/api/v1/payments/callback/m

# App
APP_URL=https://gravitypro.kvlbusinesssolutions.com
PORT=8002
NODE_ENV=production
```

Mobile — `mobile/.env`

```
EXPO_PUBLIC_API_URL=https://gravitypro.kvlbusinesssolutions.com
```

31. Project File Structure

```
Gravity/
│
├── backend/                          ← Express.js API (Port 8002)
│   ├── src/
│   │   ├── app.js                    ← Main entry, middleware, route mounting
│   │   ├── config/
│   │   │   └── db.js                 ← PostgreSQL pool (Neon connection string)
│   │   ├── middleware/
│   │   │   ├── auth.js               ← JWT authenticate() middleware
│   │   │   └── validate.js           ← Zod validate() middleware
│   │   ├── services/
│   │   │   ├── geofence.js           ← checkGeofenceStatus() helper
│   │   │   ├── sse.js                ← sendToCircleMembers() helper
│   │   │   └── payments/
│   │   │       └── index.js          ← Payment gateway factory
│   │   └── routes/
│   │       ├── auth.js               ← OTP, verify-phone, register-free,
│   │       │                            register-with-payment, Google OAuth
│   │       ├── users.js              ← Profile, location, push token, stats
│   │       ├── circles.js            ← Family group management
│   │       ├── sos.js                ← SOS alerts + push notifications
│   │       ├── geofences.js          ← Safe zones (PostGIS)
│   │       ├── locations.js          ← GPS tracking + SSE broadcast
│   │       ├── media.js              ← Cloudflare R2 upload URLs
│   │       ├── sse.js                ← SSE stream handler
│   │       ├── payments.js           ← Multi-gateway payments + webhooks
│   │       │                            + create-order-anon (pre-registration)
│   │       ├── subscriptions.js      ← Subscription status + cancel + history
│   │       └── admin.js              ← Full admin management API
│   ├── migrations/
│   │   ├── 001_initial.sql           ← Core schema (8 tables)
│   │   ├── 004_add_google_id.sql     ← google_id column on users
│   │   ├── 005_add_account_type.sql  ← account_type column on users
│   │   ├── 006_subscriptions.sql     ← subscription_plans, user_subscriptions,
│   │   │                                payment_orders tables + current_plan on users
│   │   └── 007_anon_payments.sql     ← user_id nullable, add phone + metadata
│   └── package.json
│
├── landing-react/                    ← React+Vite Web App (Port 8090)
│   ├── index.html                    ← Vite entry
│   ├── server.cjs                    ← Static file server + SPA fallback
│   ├── vite.config.ts
│   ├── src/
│   │   ├── main.tsx                  ← React root + ErrorBoundary + window.onerror
│   │   ├── index.css                 ← Global styles
│   │   └── pages/
│   │       ├── Home.tsx / .module.css  ← Landing page (simplified)
│   │       ├── Login.tsx / .module.css ← Multi-step signup + login tab switcher
│   │       ├── Pricing.tsx / .module.css ← Plan comparison page
│   │       ├── Checkout.tsx / .module.css ← Checkout flow
```

```
│   │       ├── Terms.tsx              ← Terms of service
│   │       ├── Privacy.tsx            ← Privacy policy
│   │       ├── Share.tsx              ← Share / referral page
│   │       ├── NotFound.tsx           ← 404 page
│   │       ├── AdminLogin.tsx         ← Admin login
│   │       ├── ParentPanel.tsx / .module.css  ← 5-tab parent dashboard
│   │       │                                    (no phone frame, fullscreen, logout)
│   │       ├── ChildPanel.tsx / .module.css   ← 6-tab child dashboard
│   │       │                                    (no phone frame, fullscreen, logout)
│   │       └── AdminPanel.tsx / .module.css   ← 7-tab admin + desktop sidebar
│   └── dist/                          ← Production build (Vite output)
│
├── mobile/                            ← React Native Expo App (Expo 54)
│   ├── App.js                         ← Root, NavigationContainer
│   ├── app.json                       ← Expo config (bundle ID, permissions)
│   └── src/
│       ├── navigation/
│       │   ├── RootNavigator.jsx      ← Auth gate
│       │   └── TabNavigator.jsx       ← 5-tab bottom nav
│       ├── screens/
│       │   ├── AuthScreen.jsx         ← Login + OTP
│       │   ├── MapScreen.jsx          ← Live family map (781 lines)
│       │   ├── CirclesScreen.jsx      ← Circle management
│       │   ├── SafeZonesScreen.jsx    ← Geofence viewer
│       │   ├── AlertsScreen.jsx       ← SOS + geofence alerts
│       │   └── ProfileScreen.jsx      ← User profile
│       ├── services/
│       │   ├── api.js                 ← Complete API client
│       │   ├── location.js            ← Background GPS
│       │   ├── notifications.js       ← Expo push setup
│       │   └── offlineQueue.js        ← Offline resilience
│       ├── components/                ← Shared UI components
│       └── theme/
│           └── colors.js              ← Color constants
│
├── caddy/
│   └── Caddyfile                      ← Reverse proxy + TLS config
│
├── ecosystem.config.js                ← PM2 process config (gravity-api id=58, gravity-w
│
├── GRAVITY_ARCHITECTURE.md            ← This document (v2.0)
└── PROJECT_REPORT.md                  ← Project status report
```

Summary

Area	Technology	Status
Backend API	Node.js + Express 5.2.x	Production
Database	Neon PostgreSQL 15 + PostGIS 3 (12 tables)	Production
Real-Time	Server-Sent Events (SSE)	Production
Web App	React 18 + TypeScript + Vite 8	Production
Signup Flow	Payment-gated, OTP → phone_token → register	Production
Subscriptions	Free / Family / Premium — 5 gateways	Production
File Storage	Cloudflare R2 (presigned upload)	Production
Process Mgmt	PM2 (id=58 API, id=59 Web)	Configured
Reverse Proxy	Caddy 2 (auto TLS)	Configured
SMS OTP	MSG91 (dev_otp fallback)	Integrated
Push Notify	Expo Push API	Integrated
Geofencing	PostGIS ST_Buffer + ST_Within	Production
Mobile App	React Native + Expo 54	~80% complete
Admin Panel	7 sections, desktop + mobile mode	Production

Document version: 2.0
Last updated: 20 June 2026
Repository: github.com/kamaralam1984/gravitypro
Domain: gravitypro.kvlbusinesssolutions.com
